Lab 5: Generics, MyArrayList, and Variance

Generic classes · resizable lists · `equals()`/`clone()` · invariance, covariance, contravariance

---

Alex (Aizierjiang Aiersilan)
Lab Lead, Teaching Assistant

September 28, 2026

The George Washington University
CSCI 2113 - Software Engineering | Fall 2026

© Aizierjiang Aiersilan. Free distribution and open-access.

Outline

1. Lab 5 at a Glance
2. Activity 1 — Building MyArrayList<E>
3. Activity 2 — Variance, equals(), and clone()
4. Assignment — RideShare with a Waitlist
5. Wrap-up
6. Acknowledgement

ng Alersilan / CSCI 2113

Lab 5 at a Glance

ng Aiersilan | CSCI 2113 | F

Four goals, three pieces of work

By the end of Lab 5 you can…

1. write a **generic class** with a type parameter `E`
2. use a **resizable list** instead of an array + a count
3. override `equals()` and `clone()` from `Object`
4. spot **invariance**, **covariance**, and **contravariance** in Java code

The work

Activity 1 build **MyArrayList<E>** step by step

Activity 2 **Variance.java**: see each kind of variance fail, then fix it

Assignment **RideShare** on **MyArrayList**, plus a waitlist

⚠ Due **Tue, Oct 6, 2026, 11:59 PM**

Before you start: setup and the working loop

Setup

- New directory for Lab 5.
- Test code prints with `IO.println(...)`: the class `java.lang.IO` ships with **JDK 25** [4]. ⚠ On an older JDK it will not compile.
- Activity 2 needs your Lab 4 files copied in: `User`, `Driver`, `Rider`, `Vehicle`, ...

The loop you will repeat all lab

```
$ javac *.java      # compile everything
$ java Main         # Activity 1 driver
$ java Variance     # Activity 2 driver
```

1. **Predict** what happens.
2. Compile / run; **read** the message.
3. **Explain** it, then fix it.

💡 Many steps *break on purpose*. The error is the lesson.

Compiler output on these slides is abridged to its key lines.

Activity 1 — Building MyArrayList<E>

ng Aiersilan | CSCI 2113 | F

Why a generic, resizable list?

Lab 4: array + count

- ✗ fixed capacity (**100** each)
- ✗ a count to keep in sync
- ✓ type-checked

`ObjectList` **(lecture)**

- ✓ one list for any object
- ✗ the type check moves to a *cast at run time*

`MyArrayList<E>`

- ✓ grows on demand
- ✓ `size()` replaces the count
- ✓ the *compiler* checks element types

A **type parameter** `E` is a placeholder: `MyArrayList<String>` holds only `Strings`, `MyArrayList<Driver>` only `Drivers`.

Step 1: the skeleton — and a compile error

```
public class MyArrayList<E> {
    private static final int INITIAL_CAPACITY = 100;
    private E[] list;
    private int size = 0;

    public MyArrayList() {
        list = new E[INITIAL_CAPACITY];
    }
}
```

```
MyArrayList.java:7: error: generic array creation
        list = new E[INITIAL_CAPACITY];
               ^
```

Why?

- The compiler **erases** `E` after type checking: it becomes `Object` in the bytecode [8].
- An array must know its element type *at run time*.
- So at run time there is no `E` to make an array of [2, 7].

Step 2: cast an `Object[]` — and read the warning

```
list = (E[]) new Object[INITIAL_CAPACITY];
```

```
$ javac MyArrayList.java
Note: MyArrayList.java uses unchecked or
      unsafe operations.
Note: Recompile with -Xlint:unchecked for
      details.
$ javac -Xlint:unchecked MyArrayList.java
MyArrayList.java:7: warning: [unchecked]
      unchecked cast
  required: E[]
  found:    Object[]
```

Is the cast safe? Yes, here:

- `list` is `private` — it never leaves the class.
- The class only ever stores `E` values in it.

So we silence the warning, in the **narrowest** spot:

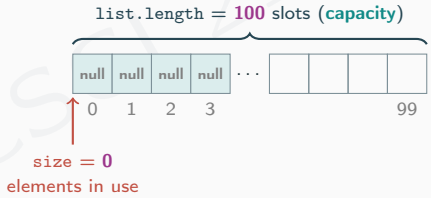
```
@SuppressWarnings("unchecked")
public MyArrayList() {
    list = (E[])
        new Object[INITIAL_CAPACITY];
}
```

The same pattern appears in [1] (Items 27, 29).

Step 3: `size()` is not `list.length`

```
public int size()        { return size; }
public boolean isEmpty() { return size == 0; }
```

```
public class Main {
    public static void main(String[] args) {
        MyArrayList<String> names =
                new MyArrayList<>();
        IO.println(names.size());    // 0
        IO.println(names.isEmpty()); // true
    }
}
```



Two different numbers

capacity: how many slots the array has.

size: how many of them hold list elements.

Step 4: `add()` and `get()` — the compiler guards the door

```
public void add(E element) {
    list[size] = element; // first free slot
    size++;
}
public E get(int index) {
    return list[index];   // no check yet
}
```

```
names.add("Ada");
names.add("Grace");
names.add("Linus");
for (int i = 0; i < names.size(); i++) {
    IO.println(names.get(i)); // Ada Grace Linus
}
```

Now try `names.add(42);`

```
error: incompatible types: int cannot be
       converted to String
```

- `MyArrayList<String>`: the mistake is caught at compile time.
- `ObjectList`: it would be accepted, and the problem shows up later, at a *cast at run time*.

→ Delete the line and move on.

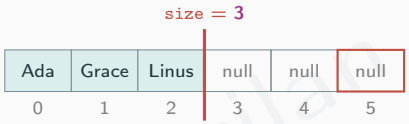
Step 5: the `get(5)` trap and `checkIndex`

With **3** names,

`IO.println(names.get(5));` prints...

```
null
```

`list[5]` exists in the *array*, but index **5** is not part of the *list*.




```
private void checkIndex(int index) {
    if (index < 0 || index >= size) {
        throw new IndexOutOfBoundsException(
            index);
    }
}
public E get(int index) {
    checkIndex(index);    // validate first
    return list[index];
}
```

Now `names.get(5)` throws:

```
java.lang.IndexOutOfBoundsException:
    Index out of range: 5
```

Valid range: **0** to `size - 1`.

Step 6: `set()` replaces — and returns what it replaced

```
public E set(int index, E element) {
    checkIndex(index);
    E old = list[index];
    list[index] = element;
    return old;
}
```

```
IO.println(names.get(1));              // Grace
IO.println(names.set(1, "Barbara"));   // Grace
IO.println(names.get(1));              // Barbara
```

- `set` never changes `size`: it only overwrites an *existing* element.
- So its valid range is the same as `get`: reuse `checkIndex`.
- 💡 Returning the old value lets the caller keep it if needed.

Step 7: `MyArrayList<int>`? And what happens at 101?

```
MyArrayList<int> numbers = new MyArrayList<>();
```

```
error: unexpected type
  required: reference
  found:    int
```

Type arguments must be **reference types** [7]. Use `Integer`: Java **boxes** an `int` into an `Integer` when you add, and **unboxes** an `Integer` wherever an `int` is needed (e.g. `int x = numbers.get(0);`) [2].

```
MyArrayList<Integer> numbers =
        new MyArrayList<>();
for (int i = 0; i < 250; i++) {
    numbers.add(i);
}
IO.println(numbers.size());
```

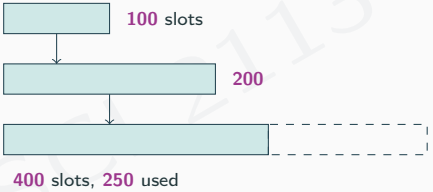
```
Exception in thread "main"
 java.lang.ArrayIndexOutOfBoundsException:
  Index 100 out of bounds for length 100
  at MyArrayList.add(MyArrayList.java:...)
  at Main.main(Main.java:...)
```

⚠ The **101**st add runs off the **100**-slot array.

Step 8: `grow()` — double the array when it is full

```
@SuppressWarnings("unchecked")
private void grow() {
    E[] bigger = (E[]) new Object[list.length * 2];
    for (int i = 0; i < size; i++) {
        bigger[i] = list[i];
    }
    list = bigger;           // old array is dropped
}

public void add(E element) {
    if (size == list.length) {
        grow();
    }
    list[size] = element;
    size++;
}
```



- **250** adds ⇒ grows **2** times: **100** → **200** → **400**.
- `numbers.size()` prints **250**; `numbers.get(249)` prints **249**.
- Same cast + `@SuppressWarnings` as the constructor.

Doubling keeps copying rare: the *average* cost per add stays constant.

Step 9: `add(index, element)` — insert and shift right

```
public void add(int index, E element) {
    if (index < 0 || index > size) { // > not >=
        throw new IndexOutOfBoundsException(index);
    }
    if (size == list.length) {
        grow();
    }
    for (int i = size; i > index; i--) { // from END
        list[i] = list[i - 1];
    }
    list[index] = element;
    size++;
}
```

Why not `checkIndex`?

`index == size` is valid here: it means "add at the end". `checkIndex` would reject it.

Order of work

1. own range check: `0 <= index <= size`
2. grow if full
3. shift, starting from the *last* element
4. store, then `size++`

Why shift from the end? Try it the wrong way

`names` = [Ada, Barbara, Linus]; now `names.add(0, "Ken")`

✗ Loop from `index` toward the end

`list[i + 1] = list[i], i = index, ..., size-1`

Each step copies the value it *just* wrote:

`[Ken, Ada, Ada, Ada]`

Barbara and Linus are overwritten before they move.

✓ Loop from the end toward `index`

`list[i] = list[i - 1], i = size, ..., index+1`

Each value moves into a slot that is already free or already copied:

`[Ken, Ada, Barbara, Linus]`

💡 Rule of thumb: shift right ⇒ loop right-to-left; shift left ⇒ loop left-to-right.

Trace it, then remove the duplication

Predict first (after `set(1, "Barbara")`):

call	names
start	[Ada, Barbara, Linus]
`add(0, "Ken")`	[Ken, Ada, Barbara, Linus]
`add(2, "Alan")`	[Ken, Ada, Alan, Barbara, Linus]

Both `add`s store and do `size++`.

So `add(E)` becomes one call:

```
public void add(E element) {
    add(size, element); // at the end
}
```

✔ grow and range logic now live in one place

CSCI 2113 Aiersilan

Step 10: `remove(index)` — shift left, then clean up

```
public E remove(int index) {
    checkIndex(index);
    E removed = list[index];
    for (int i = index; i < size - 1; i++) {
        list[i] = list[i + 1]; // left to right
    }
    size--;
    list[size] = null;   // drop stale reference
    return removed;
}
```

```
IO.println(names.remove(0)); // Ken
// now [Ada, Alan, Barbara, Linus]
```

Why `list[size] = null`?

After `size--`, slot `list[size]` is outside the list but still holds a leftover reference. Once that object leaves the list, the stray slot would keep it reachable, so the garbage collector could not reclaim it. Clearing it keeps the array referring only to list elements.

This is the "obsolete object reference" problem in [1] (Item 7).

MyArrayList<E>: the contract you must meet

Method	Does	Valid index
`MyArrayList()`	empty list, capacity **100**	—
`void add(E)`	append at end (grows)	—
`void add(int, E)`	insert, shift right	$0 \le$ `index` $\le$ `size()`
`E get(int)`	read	$0 \le$ `index` $<$ `size()`
`E set(int, E)`	replace, return old	$0 \le$ `index` $<$ `size()`
`E remove(int)`	delete, shift left, return it	$0 \le$ `index` $<$ `size()`
`int size()`	number of elements	—
`boolean isEmpty()`	`size() == 0`	—
`void addAll(MyArrayList<? extends E>)`	append all (Activity 2)	—
`MyArrayList<E> clone()`	shallow copy (Activity 2)†	—

Outside the valid range: `throw new IndexOutOfBoundsException(index)`.

→ Only `add(int, E)` accepts `index == size()`.

All methods are `public`. †Declared `throws CloneNotSupportedException`.

Activity 2 — Variance, equals(), and clone()

ng Aiersilan | CSCI 2113 | F

Three words for one question

Given `Driver` is a subtype of `User` (a `Driver` *is a* `User`), does a type built from `Driver` follow that relation, ignore it, or reverse it?

Kind	Where you meet it in Lab 5	Checked
Invariant: ignores it	`MyArrayList<Driver>` and `MyArrayList<User>` are unrelated	compile time
	overriding method: parameter types must match exactly	compile time
Covariant: follows it	arrays: `Driver[]` $\leq$ `User[]`	run time
	overriding method may return a subtype (`clone`)	compile time
	`MyArrayList<Driver>` $\leq$ `MyArrayList<? extends User>`	compile time
Contravariant: reverses it	`MyArrayList<User>` $\leq$ `MyArrayList<? super Driver>`	compile time

$A \leq B$: a value of type A can be used where a B is expected.

➔ Setup: copy your Lab 4 files next to `MyArrayList.java`, then create `Variance.java`. (`Main.java` can be archived.)

Invariance: a list of drivers is not a list of users

```
MyArrayList<Driver> drivers = new MyArrayList<>();
drivers.add(new Driver("Ada", 3, true, 4.8));
drivers.add(new Driver("Linus", 0, true, 4.1));

MyArrayList<User> users = drivers;    // ?
```

```
error: incompatible types: MyArrayList<Driver>
       cannot be converted to MyArrayList<User>
```

Suppose it were allowed

```
users.add(new Rider("Ana", 5.0));
Driver d = drivers.get(2);  // a Rider!
```

`users` and `drivers` are the *same* object, so a `Rider` would sit inside a list of `Drivers`.

Generic types are **invariant** [5]: the compiler blocks the assignment. (Delete the line.)

The `equals` trap: same objects, different answers

```
Driver a = drivers.get(0);
Driver b = new Driver("Ada", 3, true, 4.8);
Object o = b;
IO.println(a.equals(b));   // true
IO.println(a.equals(o));   // false (!)
```

`o` and `b` refer to the *same* object.

Lab 4's `Driver` has `equals(Driver other)` — an **overload**, not an override. Now `Driver` has two:

call	arg declared as	runs
`a.equals(b)`	`Driver`	`equals(Driver)` (name & zone)
`a.equals(o)`	`Object`	`Object.equals` (`a == o`)

The compiler picks the overload from the declared type of the argument. `equals(Driver)` cannot accept an `Object`, so `a.equals(o)` runs the `equals(Object)` that `Driver` inherits from `Object`. It checks *referential equality*: `a` and `o` are different objects ⇒ `false`.

ng Aiersilan | CSC 2113

`@Override` tells the truth

```
@Override
public boolean equals(Driver other) { ... }
```

```
error: method does not override or implement
       a method from a supertype
```

When overriding, Java requires the parameter types to match exactly [2]: parameters are **invariant**. A method with a different parameter type is a new *overload*, so `@Override` reports an error.

Narrower vs. wider parameters

`equals(Driver)` is *narrower* than `equals(Object)`: it would refuse arguments the parent accepts — unsafe even in theory.

A *wider* parameter (**contravariance**) would be safe in theory: every argument the parent accepts is still accepted.

✗ Java does not allow it either: a different parameter type is just another overload.

💡 Put `@Override` on every method you mean to override.

Overriding `equals(Object)` in four steps

```
@Override
public boolean equals(Object other) {
    if (this == other) {            // 1. same object?
        return true;
    }
    if (!(other instanceof Driver)) {
        return false;               // 2. wrong type
    }
    Driver that = (Driver) other;   // 3. downcast
    return getName().equals(that.getName())
        && getZone() == that.getZone(); // 4. compare
}
```

Use your own Lab 4 field or getter names.

- `a.equals(b)` and `a.equals(o)` both print `true` now.
- `instanceof` is `false` for `null`, so step 2 also handles `equals(null)`.
- Compare `String`s with `.equals`, not `==`.
- **Vehicle**: same pattern, equal when the plates match.

→ Next: `hashCode()`, which the handout now requires too.

Overriding `hashCode()` to match `equals()`

`Driver` — from the fields `equals` compares:

```
@Override
public int hashCode() {
    return 31 * getName().hashCode() + getZone();
}
```

`Vehicle` — the same way, from the plate:

```
@Override
public int hashCode() {
    return plate.hashCode();
}
```

Required: `public int hashCode()` in both classes (handout "In summary").

The contract [9]

If `a.equals(b)` is `true`, then `a.hashCode() == b.hashCode()` must hold.

The reverse is *not* required: unequal objects may share a hash code.

- Use *exactly* the fields `equals` compares: name and zone; the plate.
- Hash-based collections look an object up by its hash code first [1].
- ⚠ Not `java.util.Objects.hash(...)`: the assignment forbids `java.util`.

Covariance, the risky kind: arrays

```
User[] accounts = new Driver[2];
accounts[0] = new Driver("Ada", 3, true, 4.8);
accounts[1] = new Rider("Ana", 5.0);
// all three lines compile
```

```
Exception in thread "main"
  java.lang.ArrayStoreException: Rider
```

- Arrays are **covariant**: a `Driver[]` *is a* `User[]`, so every line compiles [2].
- The array still knows it is a `Driver[]`, and rejects the `Rider` at run time.

same mistake	`MyArrayList<Driver>` as `MyArrayList<User>`	`Driver[]` as `User[]`
caught by	✔ the compiler	✘ the running program

Safe covariance: `? extends User`

```
static double averageRating(
        MyArrayList<? extends User> users) {
    double sum = 0;
    for (int i = 0; i < users.size(); i++) {
        User u = users.get(i);    // reading: OK
        sum += u.getRating();     // Lab 4 getter
    }
    return sum / users.size();
}
```

With `MyArrayList<User>` as the parameter, passing `drivers` fails: *incompatible types*. With `? extends User`, both `drivers` and `riders` compile.

Now try writing inside it:

```
users.add(new Driver("Grace", 1, true, 5.0));
```

```
error: incompatible types: Driver cannot
       be converted to CAP#1
```

The list holds *some* subtype of `User` — the compiler does not know which (`CAP#1`). It can promise you a `User` when you `read`, but it rejects every add (except `add(null)`). (Delete the line.)

addAll: accept lists of any subtype of E

```
public void addAll(MyArrayList<? extends E> other) {
    int n = other.size();      // read size once
    for (int i = 0; i < n; i++) {
        add(other.get(i));     // an E comes out
    }
}
```

```
MyArrayList<User> users = new MyArrayList<>();
users.addAll(drivers); // MyArrayList<Driver>
users.addAll(riders);  // MyArrayList<Rider>
```

With `addAll(MyArrayList<E> other)`, both calls fail to compile (the `riders` call the same way, with `MyArrayList<Rider>`):

```
error: incompatible types:
  MyArrayList<Driver> cannot be converted
  to MyArrayList<User>
```

`other` only produces values for us to read, so `? extends E` is the right type.

💡 Reading the size once makes `list.addAll(list)` copy each element once, instead of chasing a size that keeps growing.

clone(): covariant return types

```
public class MyArrayList<E> implements Cloneable {
    @Override
    public Object clone()
            throws CloneNotSupportedException {
        return super.clone();
    }
}
// main: add "throws CloneNotSupportedException"
MyArrayList<Driver> copy = drivers.clone();
```

```
error: incompatible types: Object cannot be
       converted to MyArrayList<Driver>
```

Change the return type and downcast:

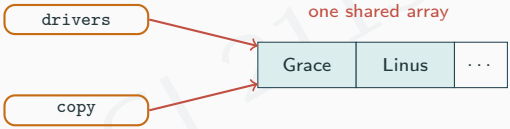
```
@Override
public MyArrayList<E> clone()
        throws CloneNotSupportedException {
    return (MyArrayList<E>) super.clone();
}
```

An override may return a *subtype* of the parent's return type: return types are **covariant** [2]. The cast cannot be checked at run time (erasure), so javac warns; the handout suppresses it on the `super.clone()` line (next slide).

`Object.clone` throws `CloneNotSupportedException` if the class does not implement `Cloneable` [9].

Shallow copy, level 1: one array, two lists

```
copy.set(0, new Driver("Grace",1,true,5.0));
IO.println(drivers.get(0)); // Grace!
```



`super.clone()` copies each *field*. `list` is a reference, so both lists point to the same array [9]. `size` is an `int`, so each list has its own.

Fix: give the copy its own array (the handout's version).

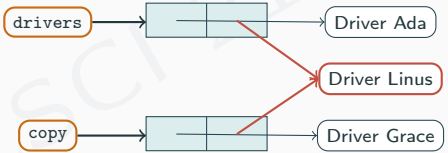
```
@Override
public MyArrayList<E> clone()
        throws CloneNotSupportedException {
    @SuppressWarnings("unchecked")
    MyArrayList<E> copy = (MyArrayList<E>) super.clone();
    copy.list = list.clone();
    return copy;
}
```

Why does only the `super.clone()` line warn? `super.clone()` returns `Object`; the cast to `MyArrayList<E>` cannot check `E`. An array's `clone()` already returns `E[]` [2]: no cast, nothing unchecked. The annotation sits on the local variable's declaration (annotations go on declarations, not statements).

Shallow copy, level 2: two arrays, same drivers

```
copy.get(1).setAvailable(false);
IO.println(drivers.get(1)); // unavailable
```

The arrays are separate now, but both arrays refer to the same `Driver` objects. Changing a driver through one list is visible through the other: `clone()` is still a **shallow copy**.



After the fix, `copy.set(0, Grace)` changes only `copy`; slot 1 of both arrays still points to the same Linus.

Scope

Using `clone()` for a *deep* copy of a generic class is outside the scope of this course and often impractical [1].

Contravariance: `? super Driver`

```
static void addAvailable(
        MyArrayList<Driver> from,
        MyArrayList<? super Driver> to) {
    for (int i = 0; i < from.size(); i++) {
        Driver d = from.get(i);
        if (d.isAvailable()) {  // Lab 4 getter
            to.add(d);          // writing: OK
        }
    }
}
```

```
MyArrayList<User> recipients = new MyArrayList<>();
addAvailable(drivers, recipients);
```

With `MyArrayList<Driver>` `to`:
incompatible types — `MyArrayList<User>` is not a `MyArrayList<Driver>`.

With `? super Driver`, `to` may be a list of `Driver`, `User`, or `Object`:
any list a `Driver` can be stored in.

try	result
pass a `MyArrayList<Rider>`	✗ compile error
`Driver d = to.get(0);`	✗ compile error
`Object d = to.get(0);`	✓ compiles

It might be a `MyArrayList<Object>`: reading only promises an `Object`.

Which wildcard? Ask what the parameter does

Parameter type	Accepts lists of	`get` gives	`add` takes	Lab 5 use
`MyArrayList<User>`	`User` only	`User`	`User`	—
`MyArrayList<? extends User>`	`User`, `Driver`, `Rider`	`User`	✗ nothing*	`averageRating`; `addAll`†
`MyArrayList<? super Driver>`	`Driver`, `User`, `Object`	`Object`	`Driver`	`addAvailable`

*except `null` [6] †with `E` in place of `User`: `? extends E`

Rule of thumb — *Producer `extends`, Consumer `super`* [1]

You only `get` values out ⇒ `? extends T`. You only `put` values in ⇒ `? super T`. Both ⇒ no wildcard. (The *Get and Put Principle* [3]; the "in/out" guideline [6].)

⚠ Delete `Variance.java` before the assignment: it is not part of the application.

Assignment — RideShare with a Waitlist

ng Aiersilan | CSCI 2113 | F

Assignment 1: replace every data array with `MyArrayList`

```
public class RideShare {
    private MyArrayList<Driver> drivers =
            new MyArrayList<>();
    private MyArrayList<Rider> riders =
            new MyArrayList<>();
    ...
}
```

- ✗ count fields → ✓ `size()`; no more cap of 100.
- Same for the rides you record — in `RideShare` or a `RideLog`.
- `DISTANCE` *stays* an `int[][]`: a fixed table of primitives.

Methods that returned arrays

method	returns
`allUsers()`	`MyArrayList<User>`
`allVehicles()`	`MyArrayList<Vehicle>`
`availableDrivers()`	`MyArrayList<Driver>`

`allUsers()`: every driver, *then* every rider — use `addAll`.

`allVehicles()`, `availableDrivers()`: in sign-up order.

Updating `Console` (and reusing `addAll`)

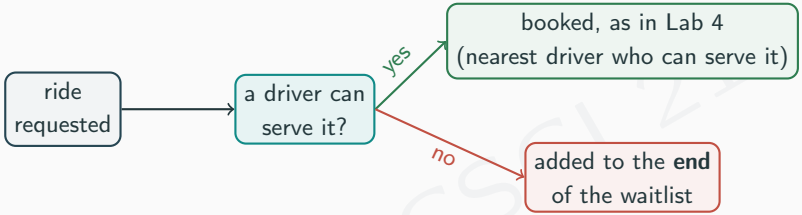
Lab 4 (array)	Lab 5 (`MyArrayList`)
`arr.length` / `count`	`list.size()`
`arr[i]`	`list.get(i)`
`count == 0`	`list.isEmpty()`

⚠ Apart from the changes described in section 2 (option **9**, the new message, and bookings after a sign-up or toggle), the console behaves exactly as in Lab 4.

```
public MyArrayList<User> allUsers() {
    MyArrayList<User> all = new MyArrayList<>();
    all.addAll(drivers);  // drivers first
    all.addAll(riders);   // then riders
    return all;
}
```

→ This is why `addAll` takes `MyArrayList<? extends E>`: with plain `MyArrayList<E>`, neither line compiles.

Assignment 2: a ride nobody can serve now *waits*



- Console prints `No driver available, added to the waitlist` (instead of Lab 4's `No driver available`).
- A ride in the waitlist is not booked: it is not in the list of rides, its `driver` is `null`, and it has no ride id yet.
- A ride in the waitlist and a booked ride are the *same* `Ride` object at different times. `Ride` now also implements `public boolean isBooked()`.
- The waitlist must be a **MyArrayList**.

When a driver becomes available

Triggers: sign-up (option **1**); toggle unavailable → available (option **5**).

Then scan the waitlist *from the front*; the *first* ride with `canBeServedBy`:

1. is removed; the others keep their order
2. gets the *next* ride id (same sequence)
3. driver → unavailable; ride recorded as booked

Not Lab 4's rule

At most **1** ride per driver this way. No nearest-driver search: the driver who became available is the *only* candidate. If they cannot serve any ride in the waitlist, nothing else happens.

Printed on the line after `Driver added` or the `... is now AVAILABLE` line, in the same format as a normal booking:

```
Your choice: 5
Driver id: 4
Linus is now AVAILABLE
Booked Ride #4 (XL): Ana with Linus | 5 miles | $12.25
```

One possible design (the method names are yours)

```
private MyArrayList<Ride> waitlist = new MyArrayList<>();

// Called by signUpDriver, and by toggleAvailability
// when the driver becomes AVAILABLE.
// Returns the ride it booked, or null.
public Ride serveWaitlist(Driver driver) {
    for (int i = 0; i < waitlist.size(); i++) {
        Ride ride = waitlist.get(i);
        if (ride.canBeServedBy(driver)) {
            waitlist.remove(i);   // keeps order
            assign(ride, driver); // id, driver, record
            return ride;          // at most one
        }
    }
    return null;
}
```

- → `remove(i)` from Activity 1 does the order-preserving removal for you.
- One booking helper (`assign`) for both paths, so ids come from one sequence.
- Lab 4's `Console` toggled with `driver.setAvailable`. Move the toggle into `RideShare` (`toggleAvailability(driver)`), so it can serve the list and return the booked `Ride`.
- `signUpDriver` must still return the `Driver`, so `Console` looks the booking up afterwards (`latestRideOf(driver)`).

This is the design used in `src/assignment`; the handout leaves the design to you.

Option 9: list the waitlist

```
---
Main Menu
(1) Sign up a driver
(2) Sign up a rider
(3) List users
(4) List vehicles
(5) Toggle a driver's availability
(6) List available drivers
(7) Request a ride
(8) List rides
(9) List ride waitlist
(q) Quit
```

Front to back, one per line:

```
Waiting (Luxury): Alan | 0 miles | $15.00
Waiting (XL): Ana | 5 miles | $12.25
```

or, if empty:

```
No rides in the waitlist
```

A ride in the waitlist prints its type, the rider's name, the travel miles, and the fare. A ride's output when *booked* is unchanged from Lab 4.

No `instanceof`: let the ride's own state decide the format.

The reference session, part 1 (steps 1–13)

#	action	result
1	list users, list the waitlist	none of either
2–6	sign up Ada (id 1, zone 3, 4 seats, not luxury), Grace (id 2, zone 2, 4 seats, luxury), rider Ana (id 3), Linus (id 4, zone 0, 7 seats, not luxury), rider Alan (id 5)	—
7	Ana: Luxury for 2, zone 0 → 4	Grace (only luxury) · #1: $2 \times (2.50 + 1.25 \times 14) =$ **$40.00**
8	Alan: Standard for 2, zone 2 → 3	Ada (4 mi from pickup; Linus 9) · #2: $2.50 + 1.25 \times 4 =$ **$7.50**
9	Ana: XL for 6, zone 1 → 4	Linus (only $\geq$6 seats) · #3: $3.50 + 1.75 \times 11 =$ **$22.75** all three drivers now unavailable
10	Alan: Luxury for 2, zone 2 → 2	waits: Alan (Luxury)
11	Ana: XL for 5, zone 3 → 4	waits: Alan (Luxury), Ana (XL)
12	Alan: Standard for 1, zone 0 → 1	waits: Alan (Luxury), Ana (XL), Alan (Standard)
13	list the waitlist	those three, in that order

The reference session, part 2 (steps 14–23)

#	action	result
14	Linus toggled available	skips Luxury (no luxury car) → serves Ana (XL): #4, 3.50+1.75×5 = **$12.25**
15	list the waitlist	Alan (Luxury), Alan (Standard)
16	Ada toggled available	skips Luxury → serves Alan (Standard): #5, 2.50 + 1.25 × 5 = **$8.75**
17	Ada toggled available again	cannot serve Luxury → nothing booked; Ada stays available
18	list the waitlist	Alan (Luxury)
19	sign up Barbara (id 6, zone 1, 4 seats, luxury)	serves the Luxury ride: #6, 2 × (2.50 + 1.25 × 0) = $5.00 → **$15.00** minimum
20	list the waitlist	`No rides in the waitlist`
21	list rides	#1 to #6
22	list available drivers	Ada only
23	quit	—

Step 14 is the key test: Linus *skips* the ride he cannot serve, and the two rides around Ana's keep their order.

Other requirements

- ✗ No data arrays. Only `DISTANCE` and the array *inside* `MyArrayList` remain.
- ✗ Nothing from `java.util` — not `ArrayList`, not any other class in that package.
- ✗ No type-name checks and no `instanceof` in `Ride` or its subclasses: ride types still differ by **overriding**.
- ✓ Helper methods and extra constructors are fine.
- All `.java` files in one directory, no packages.
- You can assume no bad input will be used in grading.

Required signatures

```
Driver, Vehicle: public boolean equals(Object other),
    public int hashCode()
RideShare:
public Driver signUpDriver(String name, int zone,
    String plate, int seats, boolean luxury)
public Rider signUpRider(String name)
public MyArrayList<User> allUsers()
public MyArrayList<Vehicle> allVehicles()
public Driver findDriver(int id)
public MyArrayList<Driver> availableDrivers()
public static int distance(int from, int to)
```

`User`, `Rider`, `Ride` + subclasses: keep Lab 4's methods; `Ride` also: `public boolean isBooked()`.

Test, check, submit

1. Replay the reference session

```
# Linux/Mac
$ java Console < lab5-session-input.txt
# Windows PowerShell
PS> Get-Content lab5-session-input.txt |
      java Console
```

Your output must match `lab5-session-full.txt` character for character.

2. Run the checker (same directory as your `.java` files)

```
$ java -jar check-lab5.jar
```

3. Package and upload

```
# Linux / Windows
$ tar -czf lab5-assignment.tgz *.java
# Mac
$ tar --no-mac-metadata \
      -czf lab5-assignment.tgz *.java
```

Upload at `submit.cs.seas.gwu.edu` → CSCI 2113 → *Lab 5 – Assignment*.

⚠ Due **Tue, Oct 6, 2026, 11:59 PM**.

Common mistakes we will look for

In `MyArrayList`

- ✗ Using `list.length` where you mean `size`.
- ✗ `add(int, E)` rejecting `index == size`.
- ✗ Growing *after* the shift, not before.
- ✗ Shifting right with a left-to-right loop.
- ✗ Forgetting `list[size] = null` in `remove`.
- ✗ `clone()` sharing one array.

In the assignment

- ✗ Giving a ride an id when it *joins* the waitlist (it gets one when *booked*).
- ✗ Serving the waitlist when a driver is toggled *to unavailable*.
- ✗ Booking more than one ride from the waitlist per trigger.
- ✗ Using the nearest-driver rule for the waitlist.
- ✗ `equals(Object)` without a matching `hashCode()`.
- ✗ Leftover count fields or arrays.
- ✗ Leftover `java.util` imports from Lab 4 (e.g. `Locale` for `String.format`).

ng Aiersilan | CSCI 2113

Wrap-up

ng Aiersilan | CSCI 2113 | F

Takeaways

1. ★ **Generics are checked at compile time, then erased**: hence `(E[]) new Object[n]` and a justified `@SuppressWarnings`.
2. ★ A list has a **size** and a **capacity**; `grow`, shift, and `checkIndex` keep them honest.
3. ★ **Invariant**: `MyArrayList<Driver>` vs `MyArrayList<User>`, and overriding parameters (so write `equals(Object)` with `@Override`, plus a matching `hashCode()`).
4. ★ **Covariant**: arrays (checked at run time) and overriding return types (`clone` returns `MyArrayList<E>`).
5. ★ **Wildcards**: `? extends` to read, `? super` to write.

In one sentence

You built **MyArrayList<E>**, learned where Java lets types vary and where it refuses, and used it to give **RideShare** a waitlist.

Acknowledgement

ng Aiersilan | CSCI 2113 | F

Acknowledgement

This lab builds on the initial materials developed by **Timothy Kim**.

We gratefully acknowledge his contribution.

ng Aiersilan | CSCI 2113

References

[1] J. Bloch.
Effective Java.
Addison-Wesley, third edition, 2018.
Items 7, 10, 11, 13, 27, 28, 29, 31.

[2] J. Gosling, B. Joy, G. Steele, G. Bracha, A. Buckley, D. Smith, and G. Bierman.
The Java language specification, Java SE 25 edition.
Oracle,
https://docs.oracle.com/javase/specs/jls/se25/html/.
Sections 4.6, 4.10.3, 5.1.7–5.1.8, 8.4.2, 8.4.8.1, 8.4.8.3, 9.6.4.4, 10.5, 10.7, 15.10.1, 2025.

[3] M. Naftalin and P. Wadler.
Java Generics and Collections.
O'Reilly Media, 2006.
Section 2.4, The Get and Put Principle.

[4] OpenJDK.
JEP 512: Compact source files and instance main methods.
OpenJDK, delivered in JDK 25.
https://openjdk.org/jeps/512, 2025.

[5] Oracle.
Generics, inheritance, and subtypes — the Java tutorials, generics.
https://docs.oracle.com/javase/tutorial/java/generics/inheritance.html.

[6] Oracle.
Guidelines for wildcard use — the Java tutorials, generics.
https://docs.oracle.com/javase/tutorial/java/generics/wildcardGuidelines.html.

[7] Oracle.
Restrictions on generics — the Java tutorials, generics.
https://docs.oracle.com/javase/tutorial/java/generics/restrictions.html.

[8] Oracle.
Type erasure — the Java tutorials, generics.
https://docs.oracle.com/javase/tutorial/java/generics/erasure.html.

[9] Oracle.
Class Object (Java SE 25 & JDK 25 API specification): equals, hashCode, clone.
https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/Object.html, 2025.

Thank you!

Questions? Let's start building `MyArrayList<E>`.